ML Assignment 2 - Submission

Anant Agarwal

Student ID: 2026AJ05360

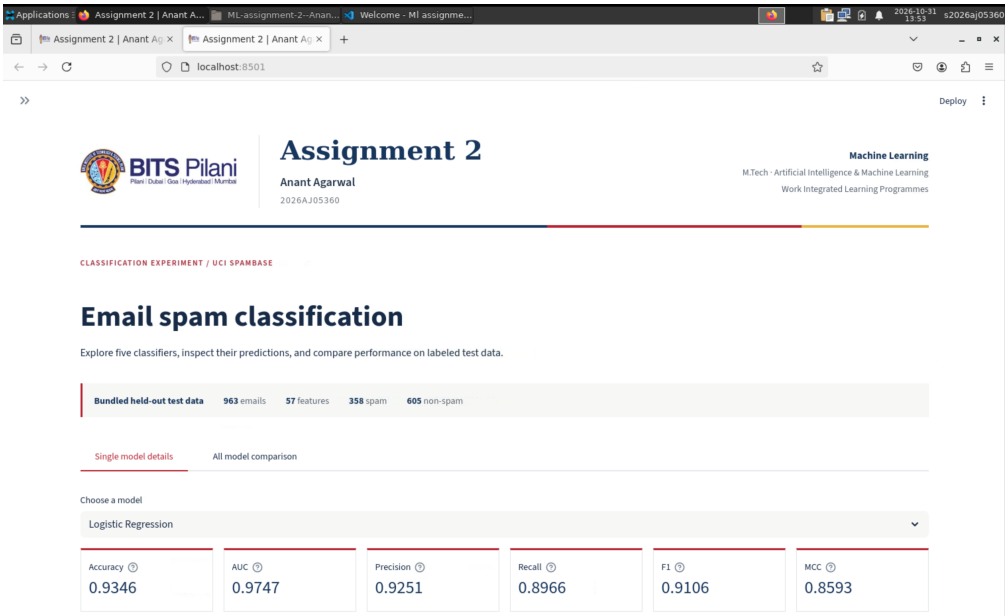
1. GitHub Repository Link

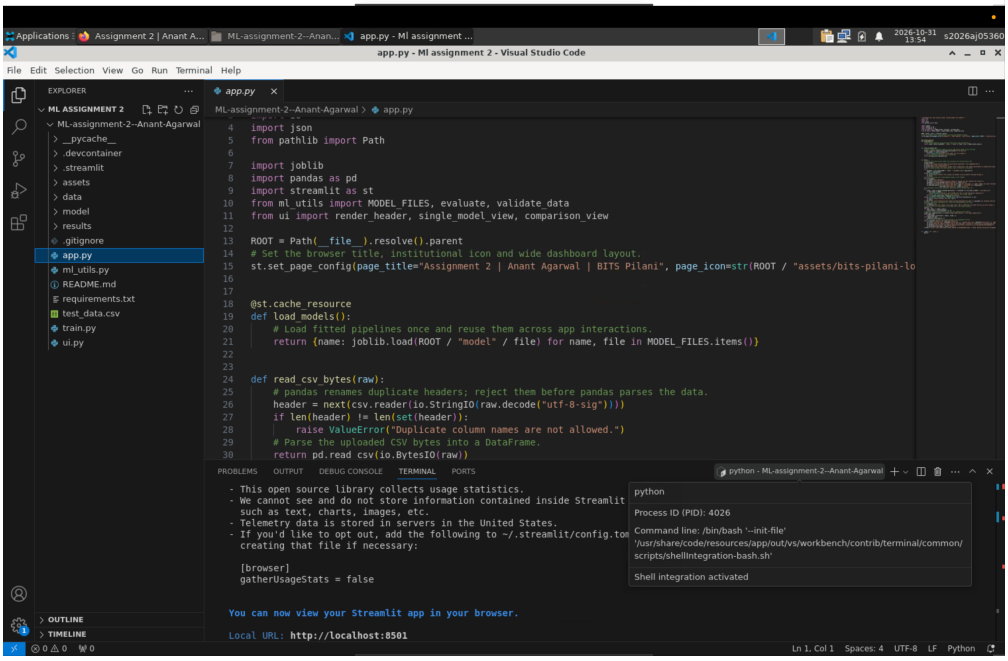
https://github.com/anantaa2000/ML-assignment-2--Anant-Agarwal.git

2. Live Streamlit App Link

https://ml-assignment-2--anant-agarwal.streamlit.app

3. BITS Virtual Lab Screenshot


Assignment 2 | Anant A...
localhost:8501
Deploy
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
Assignment 2
Anant Agarwal
2026AJ05360
Machine Learning
M.Tech · Artificial Intelligence & Machine Learning
Work Integrated Learning Programmes
CLASSIFICATION EXPERIMENT / UCI SPAMBASE
Email spam classification
Explore five classifiers, inspect their predictions, and compare performance on labeled test data.
Bundled held-out test data 963 emails 57 features 358 spam 605 non-spam
Single model details
All model comparison
Choose a model
Logistic Regression
Accuracy 0.9346
AUC 0.9747
Precision 0.9251
Recall 0.8966
F1 0.9106
MCC 0.8593


app.py - Ml assignment 2 - Visual Studio Code
File Edit Selection View Go Run Terminal Help
EXPLORER
ML ASSIGNMENT 2
ML-assignment-2--Anant-Agarwal
__pycache__
.devcontainer
.streamlit
assets
data
model
results
.gitignore
app.py
ml_utils.py
README.md
requirements.txt
test_data.csv
train.py
ui.py
OUTLINE
TIMELINE
app.py
ML-assignment-2--Anant-Agarwal > app.py
import json
from pathlib import Path
import joblib
import pandas as pd
import streamlit as st
from ml_utils import MODEL_FILES, evaluate, validate_data
from ui import render_header, single_model_view, comparison_view
ROOT = Path(__file__).resolve().parent
Set the browser title, institutional icon and wide dashboard layout.
st.set_page_config(page_title="Assignment 2 | Anant Agarwal | BITS Pilani", page_icon=str(ROOT / "assets/bits-pilani-lo
@st.cache_resource
def load_models():
Load fitted pipelines once and reuse them across app interactions.
return {name: joblib.load(ROOT / "model" / file) for name, file in MODEL_FILES.items()}
def read_csv_bytes(raw):
pandas renames duplicate headers; reject them before pandas parses the data.
header = next(csv.reader(io.StringIO(raw.decode("utf-8-sig"))))
if len(header) != len(set(header)):
raise ValueError("Duplicate column names are not allowed.")
Parse the uploaded CSV bytes into a DataFrame.
return pd.read_csv(io.BytesIO(raw))
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python - ML-assignment-2--Anant-Agarwal
- This open source library collects usage statistics.
- We cannot see and do not store information contained inside Streamlit
such as text, charts, images, etc.
- Telemetry data is stored in servers in the United States.
- If you'd like to opt out, add the following to ~/.streamlit/config.to
creating that file if necessary:
[browser]
gatherUsageStats = false
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
python
Process ID (PID): 4026
Command line: /bin/bash '--init-file'
'/usr/share/code/resources/app/out/vs/workbench/contrib/terminal/common/
scripts/shellIntegration-bash.sh'
Shell integration activated
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python

4. GitHub README

Machine Learning Assignment 2: Spam Email Classification

a. Problem statement

Predict if an email is spam (1) or not spam (0) using numeric features of email. Train Logistic Regression, Decision Tree, KNN, Gaussian Naive Bayes and Support Vector Machine on the same training partition, compare six metrics on the same held-out partition and display saved models using Streamlit.

b. Dataset description

Source: [UCI Spambase](https://archive.ics.uci.edu/dataset/94/spambase). Creators: Mark Hopkins, Erik Reeber, George Forman and Jaap Suermondt (1999). DOI: https://doi.org/10.24432/C53G6X. License: CC BY 4.0, as stated by UCI.

The dataset contains 4,601 emails and 57 input features. There are 48 word-frequency features, 6 character-frequency features, and 3 capital-letter run-length features. The target is spam, with 1,813 spam and 2,788 non-spam records. Inputs are numeric, non-negative and complete. The original feature definitions are in data/spambase.names. This app accepts these numeric features, not raw email text.

Protocol: First fold of StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42); identical feature vectors grouped. Train: 3,638 rows; test: 963 rows. The dataset has 4,207 unique feature vectors. Identical vectors, including any with conflicting labels, remain in the same partition; feature-vector overlap is zero. Grouping means the split and class proportions are approximately 80/20. All original rows are retained. Split indices and archive SHA-256 are saved for reproducibility.

StandardScaler is fitted only on training rows for Logistic Regression, KNN and SVM and saved in each pipeline. Decision Tree and GaussianNB use original feature scales. All hyperparameters were fixed in advance; this experiment does not perform hyperparameter tuning or cross-validation of model performance. The five-fold splitter only constructs one held-out partition. No test data is used for fitting or preprocessing.

c. GitHub Repository Link

https://github.com/anantaa2000/ML-assignment-2--Anant-Agarwal.git

Live Streamlit App: https://ml-assignment-2--anant-agarwal.streamlit.app

d. Models used

Logistic Regression: C=1, max_iter=3000. Decision Tree: max_depth=12, min_samples_leaf=3. KNN: k=7, uniform weighting, Euclidean distance. Gaussian Naive Bayes: default variance smoothing (1e-9). SVM: RBF kernel, C=1, gamma=scale. Other parameters use scikit-learn 1.6.1 defaults; randomized estimators use seed 42.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9346	0.9747	0.9251	0.8966	0.9106	0.8593
Decision Tree	0.9242	0.9358	0.9228	0.8687	0.8950	0.8367
KNN	0.9128	0.9560	0.8848	0.8799	0.8824	0.8131
Naive Bayes	0.8255	0.9455	0.6900	0.9637	0.8042	0.6843

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Support Vector Machine	0.9315	0.9733	0.9195	0.8939	0.9065	0.8526

Accuracy is the fraction correctly classified. Precision, recall and F1 use spam (1) as the positive class and binary averaging. ROC AUC uses continuous decision scores for Logistic Regression/SVM and P(spam) for the other models, rather than hard labels. MCC summarizes agreement across all confusion-matrix cells and ranges from -1 to 1. Undefined precision/recall/F1 use zero; AUC is shown as N/A for a single-class upload. Both target classes occur in the bundled test set.

ML Model Name	Observation about model performance
Logistic Regression	Test F1=0.9106, precision=0.9251, recall=0.8966, AUC=0.9747. A standardized linear baseline. Its decision boundary cannot directly model nonlinear feature interactions. These mechanisms are possible explanations, not causal findings.
Decision Tree	Test F1=0.8950, precision=0.9228, recall=0.8687, AUC=0.9358. Captures nonlinear rules without scaling. Depth 12 and at least 3 samples per leaf limit model complexity. These mechanisms are possible explanations, not causal findings.
KNN	Test F1=0.8824, precision=0.8848, recall=0.8799, AUC=0.9560. Uses 7 neighbors after standardization. Local similarity helps, but irrelevant dimensions and sparse neighborhoods can hurt. These mechanisms are possible explanations, not causal findings.
Naive Bayes	Test F1=0.8042, precision=0.6900, recall=0.9637, AUC=0.9455. Uses Gaussian likelihoods without scaling. Correlated word frequencies and non-Gaussian features can violate its assumptions. These mechanisms are possible explanations, not causal findings.
Support Vector Machine	Test F1=0.9065, precision=0.9195, recall=0.8939, AUC=0.9733. Uses a standardized RBF kernel to capture nonlinear relationships. C=1 and gamma=scale were fixed before evaluation. These mechanisms are possible explanations, not causal findings.
Overall winner for this dataset	Logistic Regression has the highest F1 on this held-out split. This is a descriptive comparison, not proof of universal superiority; no hyperparameters were selected using these test scores.

Limitations: this older email corpus includes source-specific words and may not represent modern spam. A single split has sampling uncertainty; no statistical significance claim is made. F1 is the declared comparison criterion; a deployment may prioritize precision to avoid blocking legitimate email. Uploaded data must represent the same features and units.

5 References

[1] Mark Hopkins, Erik Reeber, George Forman and Jaap Suermondt. "UCI Spambase Data Set." UCI Machine Learning Repository, 1999.

[2] WWW.google.com for content and links.

[3] www.chatgpt.com for AI assistance in writing UI code, helper functions and for paraphrasing documentation.

[4] bits-pilani.ac.in for UI theme, Logo.

[5] www.streamlit.io for Streamlit Community Cloud deployment.

[6] scikit-learn.org for model implementations, documentation and metrics.